

# Tally — Personal Expense Tracker

A front-end-only guide to how the project works, under the hood. Companion to `Agent tasks.md`, `Codebase.md` and the PRD.

## 1. Project overview

Tally is a **single-page React app that runs entirely in the browser**. There is no backend, no database, no API — all data lives in the browser's `localStorage`, keyed per sign-in account. That is the app's core idea: private, on-device, no server.

**What it does** — track personal income & expenses, categorise them, see totals and reports, back up to a JSON file, and restore by merging. Future-dated transactions are "upcoming" and excluded from totals until their date arrives.

- **React 18 + Vite** for the build/dev server `frontend`
- **Tailwind CSS 3** for styling (utility classes, no CSS framework)
- **react-router-dom v6** for navigation
- **recharts** for charts (donut, bar, trend)
- **Radix UI primitives + shadcn-style wrappers** for dialog / toast / inputs
- **lucide-react** icons, **vitest** for unit tests

## 2. Project structure

```
src/  
├ main.jsx, App.jsx      entry point & router/providers  
├ index.css              Tailwind + theme tokens  
├ Layout.jsx            app shell (sidebar, mobile nav, banners, footer)  
├ components/           UI building blocks  
│   ├── Sidebar, PeriodPicker, TransactionForm, TransactionRow,  
│   │   EmptyState, StatCard, Section, ConfirmDialog, BackupReminder, ScrollToTop  
│   ├── charts/         CategoryDonut, IncomeExpenseBar, SpendingTrend  
│   └── ui/              kept primitives: button, dialog, input, label,  
│                       textarea, toast, toaster, use-toast  
├ pages/                SignIn, Dashboard, Add, Transactions,  
│                       Reports, Categories, Settings, Help  
├ lib/                  the "brain" – all logic & state  
│   ├── store.jsx        global state (auth, data, actions, persistence)  
│   ├── storage.js        localStorage layer (per-account keys)  
│   ├── calc.js          pure math: totals, balance, reports, trend  
│   ├── format.js        money/date formatting + uid()  
│   ├── period.jsx       shared Month/Year/All-time picker state  
│   ├── filters.jsx      persisted list filters (search/sort/etc.)  
│   ├── constants.js     names, domain, currencies, nav items  
│   ├── strings.js       every user-visible string, centralised  
│   ├── transaction-validation.js / category-delete.js / restore-merge.js = pure logic  
│   ├── google-signin.js real Google sign-in (only if a client ID is set)  
│   ├── utils.js         cn() class helper  
│   └── __tests__/        vitest tests for the pure logic  
├ hooks/                use-today, use-mobile, use-size, use-toast  
└ globals.d.ts          types for import.meta.env, window.google
```

Rule of thumb: `pages` = screens, `components` = reusable pieces, `lib` = logic/state, `hooks` = React behaviour.

## 3. Startup flow — what happens first

```

index.html → main.jsx →
App      → Router + ScrollToTop
        → <AppProvider>      (creates the global state)
          → <Gate/>           (waits for load, shows spinner)
            → no user? → <SignIn/>
            → user?   → PeriodProvider + FiltersProvider
                      → Layout (sidebar) → Routes

```

**Providers in a sentence:** `AppProvider` owns all data & action functions. `PeriodProvider` shares which month/year everyone is looking at. `FiltersProvider` keeps the Transactions page's search/filters alive when you navigate away and back.

On start, `store.js` reads the saved session ( `tally:session` ). If a user is remembered, it loads **their** data and shows the dashboard; otherwise you land on the sign-in screen.

## 4. The state layer — `store.js` (the heart)

Everything important lives in one React context via the `useApp()` hook. Pages never touch `localStorage` directly — they call actions and the store persists for them.

| Piece                     | What it holds                                                                          |
|---------------------------|----------------------------------------------------------------------------------------|
| <code>user</code>         | current signed-in account { <code>id</code> , <code>name</code> , <code>email</code> } |
| <code>transactions</code> | list of records (amounts are <i>integer hundredths</i> )                               |
| <code>categories</code>   | predefined + user-added expense categories                                             |
| <code>settings</code>     | display currency, last backup date, backup snooze                                      |
| <code>corruptKinds</code> | data kinds whose stored value failed to parse (protects it)                            |

**Actions** you'll find: `signIn` / `signOut`, `add/update/deleteTransaction`, `add/rename/deleteCategory`, `updateSettings`, `resetAllData`, `createBackup`, `restoreBackup` (preview only), `applyRestore` (commit).

## 5. Core rules that explain most behaviour

- **Amounts are stored as integer hundredths** (e.g. £12.34 → `1234` ). Avoids floating-point money bugs.
- **Counted vs upcoming** — a transaction is "counted" when its date is today or earlier. Totals, balance and charts only use counted ones. Future-dated ones appear as an "X upcoming, not counted yet" notice.
- **Per-account data** — keys look like `tally:user-id:transactions` . Two accounts on one browser never mix.
- **Every change auto-saves** — if a save fails (storage full/blocked) you get a toast, never a silent loss.
- **Corrupt data is never overwritten** — unreadable stored data stays untouched on disk, shows a warning banner, and points you to Settings → Restore.
- **Multi-tab sync** — the `storage` event keeps all open tabs in sync automatically.

## 6. Frontend journey — screen by screen

### Sign-in ( `pages/SignIn.jsx` )

By default it's a *simulated* Google picker ( `DEMO_ACCOUNTS` ) plus a manual "use another account" form. Only `@northwind.co` emails are allowed (one config value: `COMPANY_DOMAIN` ). When you set the env var `VITE_GOOGLE_CLIENT_ID` , the page switches to the *real* Google Identity Services flow instead — nothing else changes.

Dashboard ( / )

Welcome card, period totals (income / expenses / net), total balance, spending-by-category donut, and recent transactions. Everything respects the shared period picker and the counted rule.

Add ( /add ) & the form ( TransactionForm.jsx )

Type toggle (expense/income), amount, category chips, date, note. Validation is a **pure function** ( transaction-validation.js ) — amount format, >0, ≤ 9999999999.99, note ≤ 100 chars. "Save & add another" keeps the dialog open.

Transactions ( /transactions )

Searchable/filterable list with type & category filters, sort, "All time", and "Load more" pagination (50 at a time). Its state lives in FiltersProvider , so it survives navigation; a page reload resets it.

Reports ( /reports )

Totals, income-vs-expense bar chart, category breakdown and a daily (month view) or monthly (year view) spending trend. Counted transactions only — that's why the "no counted transactions" hint exists.

Categories ( /categories )

Add / rename / delete your own expense categories (max 30 chars, unique). Built-ins can't be touched. Deleting a category *moves its transactions to "Other"* — that logic lives in category-delete.js .

Settings ( /settings )

Display currency (label-only, no conversion), categories shortcut, backup / restore, account + sign out, and "Reset all data" (with type-RESET confirmation).

Help ( /help )

Plain-language explanation of where data lives and how to back up.

7. Backup & restore (the interesting part)

Backup downloads a JSON file: { format:"tally-backup", version, currency, categories, transactions } .

Restore is **two-phase**: pick a file → it parses & computes a preview (X added / Y skipped / Z updated) *without changing anything* → you confirm → then it commits. The merge rules (in restore-merge.js ):

- Transactions merge **by id**; a newer updatedAt wins, matching ones are skipped.
- Categories merge by lower-cased type:name ; brand-new names are added as custom categories and incoming transactions are re-pointed to the right ids.

8. Persistent storage & safety nets

- requestPersistentStorage() asks the browser to keep the site's data (reduces eviction) — one-shot, best-effort.
- Pending data isn't committed on the restore dialog unless you confirm (BAK-04 compliant).
- If stored data can't be parsed, the app shows a banner, keeps the raw value, and lets you restore or reset (ARC-06).

9. Testing & quality checks

| Command     | What it does                       |
|-------------|------------------------------------|
| npm run dev | dev server → http://localhost:5173 |

|                                |                                                                           |
|--------------------------------|---------------------------------------------------------------------------|
| <code>npm test</code>          | Vitest — 37 unit tests (calc, validation, category-delete, restore-merge) |
| <code>npm run lint</code>      | ESLint check                                                              |
| <code>npm run build</code>     | production build to <code>dist/</code>                                    |
| <code>npm run preview</code>   | serve the built app locally                                               |
| <code>npm run typecheck</code> | TypeScript check (40 pre-existing, unrelated errors)                      |

**Why the tests work without a browser:** the tricky logic (money math, validation, merge, category-delete) was extracted into plain functions in `src/lib` that don't touch React or the DOM — so `vitest` tests them directly.